

TEMA 06

Manejo de formularios

Desarrollo Web en Entorno Servidor
Tema 6 · Curso 2026/2027

[Conceptos](#) · [Código](#) · [Ejemplos](#) · [Actividades](#)

Qué aprenderás

- Crear formularios HTML con controles adecuados y etiquetas accesibles.
- Elegir GET o POST y recuperar los datos desde PHP.
- Validar las entradas y mostrar resultados en HTML de forma segura.
- Recibir archivos y comprobar sus errores, tamaño y tipo.
- Construir enlaces con parámetros y redirigir después de procesar un envío.

Requisitos: HTML, variables, arrays, condiciones y funciones. Ejemplos para PHP 8.x.

Del navegador al servidor

- **1. Formulario:** la persona introduce los datos y pulsa Enviar.
- **2. Petición HTTP:** el navegador envía los campos al destino indicado en action.
- **3. PHP:** el servidor recibe, valida y procesa los datos.
- **4. Respuesta:** devuelve HTML o una redirección que el navegador interpreta.

El navegador recibe la respuesta generada; el código PHP se ejecuta en el servidor.

Prepara el entorno

- Guarda los archivos en UTF-8. Usa una carpeta distinta para cada práctica.
- Ejecuta PHP en un servidor local, como el configurado en clase.
- Alternativa: `php -S 127.0.0.1:8000` desde la carpeta de trabajo.
- Abre `http://127.0.0.1:8000/formulario.html` y revisa la pestaña Red del navegador.

El servidor integrado es solo para desarrollo. Los fragmentos PHP se guardan con `<?php` al principio, salvo que ya se muestre.

Un formulario mínimo

```
<form action="saludo.php" method="post">  
  <label for="nombre">Tu nombre</label>  
  <input id="nombre" name="nombre" type="text" required>  
  <button type="submit">Enviar</button>  
</form>
```

- action indica el destino; method indica cómo se envían los datos.
- label se asocia al control mediante for e id.

Fragmento para el body de una página HTML con lang="es", título y <meta charset="utf-8">.

name, id y value

Atributo	Para qué sirve	Ejemplo
name	Nombre enviado al servidor	name="nombre"
id	Identifica el elemento en HTML	id="nombre"
value	Valor inicial o valor de una opción	value="tarde"
type	Tipo de control input	type="email"

PHP usa name como clave. Un id no sustituye a name. <input> no tiene etiqueta de cierre.

Controles para introducir datos

Control	Uso	Detalle
text / email / url / tel	Texto y datos de contacto	tel no valida el formato
number / range	Valores numéricos	min, max y step
date / time / color	Fecha, hora y color	La interfaz varía
password / hidden	Contraseña / dato oculto	No aportan cifrado

Otros tipos: search, file, radio, checkbox y botones. El servidor valida los datos aunque el navegador los haya comprobado.

Atributos que conviene distinguir

Atributo	Efecto
required / minlength / maxlength	Restricciones del navegador para campos compatibles.
size	Anchura visual aproximada; no limita la longitud.
readonly	Impide editar controles compatibles; su valor se envía.
disabled	Desactiva el control; su valor no se envía.
placeholder	Pista breve; no sustituye a una etiqueta label.

Mantén un orden natural de tabulación. No uses tabindex positivo para reorganizar el formulario.

Texto largo y sugerencias

```
<label for="consulta">Consulta</label>
<textarea id="consulta" name="consulta" rows="4"
          cols="40" maxlength="500"></textarea>
<label for="lenguaje">Lenguaje</label>
<input id="lenguaje" name="lenguaje" list="lenguajes">
<datalist id="lenguajes">
  <option value="PHP">
  <option value="Python">
</datalist>
```

textarea contiene su valor entre las etiquetas. datalist es un elemento de sugerencias, no un tipo de input ni una lista cerrada.

Radio: elegir una sola opción

```
<fieldset>
  <legend>Turno</legend>
  <label><input type="radio" name="turno"
              value="manana" checked> Mañana</label>
  <label><input type="radio" name="turno"
              value="tarde"> Tarde</label>
</fieldset>
```

- El mismo name agrupa las opciones; value identifica la elegida.

checked fija la selección inicial. El servidor debe comprobar que el valor recibido pertenece a las opciones permitidas.

Checkbox: elegir varias opciones

```
<fieldset>
  <legend>Intereses</legend>
  <label><input type="checkbox" name="intereses[]"
           value="php"> PHP</label>
  <label><input type="checkbox" name="intereses[]"
           value="html"> HTML</label>
</fieldset>
```

- Con [] en el nombre, PHP agrupa los valores en un array.

Las casillas sin marcar no se envían. Si no se marca ninguna, la clave intereses puede no existir.

Select: una lista cerrada

```
<label for="ciudad">Ciudad</label>
<select id="ciudad" name="ciudad" required>
  <option value="" selected>Elige una ciudad</option>
  <option value="sevilla">Sevilla</option>
  <option value="granada">Granada</option>
</select>
```

- Se envía el value de la opción seleccionada.

Para selección múltiple: multiple y name="ciudades[]". size controla cuántas opciones se ven a la vez.

Botones y campos especiales

Control	Comportamiento
<code>button type="submit"</code>	Envía el formulario. Indica siempre el tipo.
<code>button type="reset"</code>	Restaura los valores iniciales; no siempre vacía.
<code>button type="button"</code>	Sin acción propia: suele usarse con JavaScript.
<code>input type="password"</code>	Oculto visualmente el texto escrito.
<code>input type="hidden"</code>	No se ve en la página, pero puede manipularse.

Un campo oculto nunca demuestra permisos ni un precio válido. Para proteger el transporte se utiliza HTTPS.

GET y POST

Aspecto	GET	POST
Dónde van los campos	En la consulta de la URL	En el cuerpo de la petición
Uso habitual	Buscar, filtrar, paginar	Enviar o modificar datos
PHP	\$_GET	\$_POST
Compartir la búsqueda	La URL conserva los parámetros	La URL no conserva el cuerpo
Confidencialidad	No cifra los datos	Tampoco cifra los datos

Evita datos sensibles en la URL. HTTPS protege el transporte de ambos métodos; POST por sí solo no lo hace.

Qué significa enctype

- **application/x-www-form-urlencoded:** codificación habitual de los campos.
- **multipart/form-data:** se utiliza con POST para adjuntar archivos.
- El navegador serializa los nombres y valores; PHP construye los arrays correspondientes.
- Usa una URL de servidor en action. Un enlace mailto no sustituye al procesamiento con PHP.

enctype describe el formato del cuerpo enviado, no un método de encriptación. En estas prácticas trabajamos en UTF-8.

Ejemplo GET: buscar productos

```
<form action="buscar.php" method="get">
  <label for="q">Buscar</label>
  <input id="q" name="q" type="search">
  <button type="submit">Buscar</button>
</form>
```

- Al escribir teclado, el navegador solicita buscar.php?q=teclado.
- buscar.php recupera la entrada con `$_GET["q"]` ?? "".

El parámetro es visible y modificable. No uses GET para acciones como borrar un registro.

Recibir no significa confiar

- Usa `$_GET` o `$_POST` según el origen esperado. `$_FILES` contiene los archivos.
- El operador `??` permite dar un valor por defecto si una clave no existe.
- Comprueba el tipo: una entrada esperada como texto también puede llegar como array.
- Valida reglas del problema y escapa el texto al insertarlo en HTML.

Evita `$_REQUEST`: puede mezclar GET, POST y cookies según la configuración, dificultando saber de dónde procede el dato.

Saludo: formulario.html

```
<!doctype html>
<html lang="es">
<meta charset="utf-8">
<title>Saludo</title>
<form action="saludo.php" method="post">
  <label for="nombre">Nombre</label>
  <input id="nombre" name="nombre" required maxlength="60">
  <button type="submit">Saludar</button>
</form>
</html>
```

Guarda ambos archivos en la misma carpeta. Este documento usa la omisión permitida de las etiquetas head y body.

Saludo: validar la entrada

```
<?php
header("Content-Type: text/html; charset=utf-8");
if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    http_response_code(405);
    exit("Envía el formulario.");
}
$nombre = $_POST["nombre"] ?? "";
if (!is_string($nombre) || trim($nombre) === "") {
    http_response_code(400);
    exit("Escribe un nombre.");
}
$nombre = trim($nombre);
```

Primera parte de saludo.php. is_string evita tratar un array como texto. El código continúa en la siguiente diapositiva.

Saludo: construir la respuesta

```
if (mb_strlen($nombre, "UTF-8") > 60) {  
    http_response_code(400);  
    exit("El nombre no puede superar 60 caracteres.");  
}  
$seguro = htmlspecialchars(  
    $nombre, ENT_QUOTES | ENT_SUBSTITUTE, "UTF-8"  
);  
echo "<!doctype html><html lang='es'><meta charset='utf-8'>";  
echo "<title>Saludo</title><h1>Hola, $seguro</h1></html>";
```

Continuación del mismo archivo, sin otro `<?php`. `mb_strlen` requiere la extensión `mbstring`. Con Ana se muestra «Hola, Ana».

Validación y salida HTML

Operación	Objetivo	Ejemplo
Validar	Aceptar solo datos adecuados	Nombre no vacío
Normalizar	Aplicar una regla de formato	trim quita espacios extremos
Escapar HTML	Mostrar texto como texto	htmlspecialchars
Proteger el transporte	Cifrar la comunicación	HTTPS

htmlspecialchars sirve para texto HTML y atributos entre comillas; no sustituye la validación ni prepara consultas SQL.

Validar un número sin perder el cero

```
$entrada = $_POST["edad"] ?? "";  
$edad = is_string($entrada)  
    ? filter_var($entrada, FILTER_VALIDATE_INT)  
    : false;  
if ($edad === false || $edad < 0 || $edad > 120) {  
    exit("Introduce una edad entre 0 y 120.");  
}  
echo "Edad válida: " . $edad;
```

Usa `=== false`: 0 es un entero válido. Para decimales, `FILTER_VALIDATE_FLOAT`; concreta el separador aceptado.

Validar varias casillas

```
$intereses = $_POST["intereses"] ?? [];  
$permitidos = ["php", "html"];  
if (!is_array($intereses)) {  
    exit("Selección incorrecta.");  
}  
foreach ($intereses as $interes) {  
    if (!in_array($interes, $permitidos, true)) {  
        exit("Opción no permitida.");  
    }  
}  
// Ahora los valores pertenecen a la lista permitida.
```

Con ninguna selección se obtiene []. La comparación estricta también rechaza estructuras inesperadas.

Práctica: saludo y calculadora

- **Saludo:** pide el nombre, valida y devuelve un mensaje personalizado.
- **Calculadora:** pide dos números y presenta suma, resta, producto y división en una tabla.
- Valida ambos números en PHP; acepta el cero y rechaza los campos vacíos.
- Si el segundo número es cero, muestra «No se puede dividir entre cero» en esa fila.

Pruebas mínimas: 8 y 2; 0 y 5; 5 y 0; un campo vacío; texto en lugar de un número.

Parámetros en un enlace

```
$consulta = http_build_query([
    "categoria" => "libros", "pagina" => 2
]);
$url = "catalogo.php?" . $consulta;
$href = htmlspecialchars($url, ENT_QUOTES, "UTF-8");
echo '<a href="' . $href . '">Ver catálogo</a>';
```

- La página de destino lee categoria y pagina desde \$_GET.

http_build_query codifica los parámetros; htmlspecialchars adapta la URL al atributo HTML. Los parámetros siguen siendo modificables.

Redirigir correctamente

```
<?php
// Después de validar y completar el procesamiento:
header("Location: resultado.php", true, 303);
exit;
```

- Envía la cabecera antes de cualquier HTML, echo o espacio de salida.
- exit evita que el resto del script siga ejecutándose.

Usa destinos controlados por tu aplicación. header() por sí solo no detiene PHP.

Ampliación: Post/Redirect/Get

- **POST:** se recibe el formulario y se valida la petición.
- **Procesamiento:** se realiza la operación y se comprueba su resultado.
- **303:** el servidor redirige a una página de resultado.
- **GET:** al recargar esa página, no se repite directamente el POST anterior.

La redirección no transporta \$_POST a la página siguiente. Las sesiones se estudiarán en su unidad. No sustituye al control de duplicados.

Enviar archivos: qué cambia

- El formulario necesita `method="post"` y `enctype="multipart/form-data"`.
- Un input de tipo file permite seleccionar el archivo.
- PHP recibe los campos de texto en `$_POST` y el archivo en `$_FILES`.
- El archivo llega primero a una ubicación temporal del servidor.

Revisa `file_uploads`, `upload_max_filesize` y `post_max_size` en `php.ini`. Si se supera `post_max_size`, los arrays pueden llegar vacíos.

Formulario para subir una imagen

```
<form action="subir.php" method="post"
      enctype="multipart/form-data">
  <label for="imagen">Imagen (JPEG o PNG, máximo 2 MiB)</label>
  <input id="imagen" name="imagen" type="file"
        accept="image/jpeg,image/png" required>
  <button type="submit">Subir imagen</button>
</form>
```

- accept orienta la selección; la comprobación real se hace en PHP.

Fragmento para una página HTML. Los cuatro pasos «Subir imagen» forman un único subir.php, en el orden indicado.

Qué contiene \$_FILES["imagen"]

Clave	Significado
name	Nombre original proporcionado por el cliente.
type	MIME declarado por el cliente; no es una validación.
tmp_name	Ruta temporal asignada por PHP.
size	Tamaño en bytes.
error	Código entero de error; 0 indica UPLOAD_ERR_OK.

No uses name como ruta de destino ni confíes en type. El ejemplo acepta un archivo; los campos múltiples tienen otra estructura.

Los errores no son un booleano

Código / constante	Qué significa
0 · UPLOAD_ERR_OK	Archivo recibido sin error de subida.
1 / 2 · INI_SIZE / FORM_SIZE	Supera un límite del servidor o del formulario.
3 · UPLOAD_ERR_PARTIAL	Solo llegó una parte del archivo.
4 · UPLOAD_ERR_NO_FILE	No se seleccionó ningún archivo.
6 / 7 / 8	Carpeta temporal, escritura o extensión de PHP.

Las constantes completas llevan el prefijo `UPLOAD_ERR_`. Si error no es `UPLOAD_ERR_OK`, no continúes con el archivo.

Subir imagen · 1. Revisar la petición

```
<?php
if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    http_response_code(405);
    exit("Usa el formulario.");
}
$f = $_FILES["imagen"] ?? null;
if (!is_array($f) || !is_int($f["error"] ?? null)) {
    exit("No se ha recibido un archivo válido.");
}
if ($f["error"] !== UPLOAD_ERR_OK) {
    exit("La subida no se ha completado.");
}
```

Comprueba también los límites de PHP cuando no llega ningún archivo. No intentes acceder a tmp_name tras un error.

Subir imagen · 2. Tamaño y origen

```
$tmp = $f["tmp_name"] ?? null;
$tamano = $f["size"] ?? null;
if (!is_string($tmp) || !is_uploaded_file($tmp)) {
    exit("Origen del archivo no válido.");
}
if (!is_int($tamano) || $tamano <= 0
    || $tamano > 2 * 1024 * 1024) {
    exit("La imagen debe ocupar entre 1 byte y 2 MiB.");
}
```

Continuación de subir.php. 2 MiB son $2 \times 1024 \times 1024$ bytes. Los límites del servidor deben permitir esta subida.

Subir imagen · 3. Comprobar el tipo

```
$fi = finfo_open(FILEINFO_MIME_TYPE);  
if ($fi === false) {  
    exit("No se pudo comprobar el tipo.");  
}  
$mime = finfo_file($fi, $tmp);  
finfo_close($fi);  
$tipos = ["image/jpeg" => "jpg", "image/png" => "png"];  
if ($mime === false || !isset($tipos[$mime])) {  
    exit("Solo se admiten imágenes JPEG y PNG.");  
}
```

Requiere la extensión Fileinfo. Se inspecciona el contenido; cambiar el nombre a foto.jpg no convierte un archivo en una imagen.

Subir imagen · 4. Guardar

```
$dir = dirname(__DIR__) . "/privado/subidas";  
if (!is_dir($dir) && !mkdir($dir, 0750, true)) {  
    exit("No se pudo preparar la carpeta.");  
}  
$nombre = bin2hex(random_bytes(16)) . "." . $tipos[$mime];  
if (!move_uploaded_file($tmp, $dir . "/" . $nombre)) {  
    exit("No se pudo guardar la imagen.");  
}  
echo "Imagen guardada correctamente.";
```

Supuesto: subir.php está en proyecto/publico y solo publico se sirve por HTTP. La carpeta privado queda fuera de la raíz pública.

Ampliación: límites del ejemplo

- El servidor necesita permisos para escribir en privado/subidas.
- El nombre aleatorio evita reutilizar el nombre aportado por el cliente.
- No publiques automáticamente archivos subidos ni permitas ejecutarlos.
- En una aplicación real añade permisos de usuario, cuotas y protección CSRF a las operaciones autenticadas.

Comprobar el MIME no garantiza que todo archivo sea inocuo. La autenticación, las sesiones y la gestión avanzada se trabajarán después.

Funciones para archivos y carpetas

Función	Utilidad
file_exists / is_dir	Comprobar existencia / comprobar que es carpeta.
mkdir	Crear una carpeta; comprobar el resultado.
scandir	Listar entradas; puede incluir . y ..
pathinfo	Separar partes de una ruta; no valida su contenido.
move_uploaded_file	Mover un archivo recibido por subida HTTP POST.

pathinfo puede no devolver extension si el nombre no tiene extensión. No comprueba que la ruta exista.

Práctica: ficha de una imagen

- Pide una imagen JPEG o PNG y un título descriptivo.
- Valida el título, el error de subida, el tamaño máximo de 2 MiB y el tipo real.
- Guarda con un nombre generado por el servidor; usa el título como dato descriptivo.
- Muestra una tabla con nombre original, nombre guardado, tamaño en MiB y tipo detectado.

Escapa los textos al mostrarlos. No utilices el título como ruta. No hace falta publicar la imagen para mostrar su ficha.

Comprueba antes de entregar

Prueba	Resultado esperado
Nombre vacío / solo espacios	Mensaje de validación; no saludo vacío.
Texto <code>Ana</code>	Se muestra como texto, sin aplicar negrita.
Entrada tipo nombre[]=Ana	Rechazo controlado, sin error de tipos.
Archivo sin seleccionar / demasiado grande	Mensaje; no se guarda.
Archivo de otro tipo renombrado a .jpg	Rechazo al comprobar el contenido.

Prueba también el envío correcto y el cero en los campos numéricos. Revisa URL, método, campos y respuesta en la pestaña Red.

Ideas clave

- name identifica el dato enviado; PHP lo recibe en el array correspondiente.
- GET consulta; POST se usa habitualmente para enviar o modificar datos.
- Valida en el servidor y escapa al generar HTML.
- En las subidas, comprueba errores, tamaño, tipo y destino.
- Redirige antes de generar salida y termina con exit.

Secuencia de trabajo: recibir → validar → procesar → responder. Elige casos de prueba antes de escribir el código.

Referencias y alcance

- [1] HTML: formularios y envío de datos · MDN
- [2] PHP: variables externas y formularios
- [3] PHP: htmlspecialchars
- [4] PHP: filter_var
- [5] PHP: header y redirecciones
- [6] PHP: subida de archivos mediante POST
- [7] PHP: códigos de error de subida
- [8] PHP: finfo_file

Adaptación del material original del docente. Revisión: septiembre de 2026. Ejemplos para PHP 8.x; ampliaciones señaladas.